

M2S1 | FR & NFR

Why Requirements?

- cut ambiguity → fewer reworks
- enable testing & acceptance
- anchor scope and schedule
- guide architecture decisions

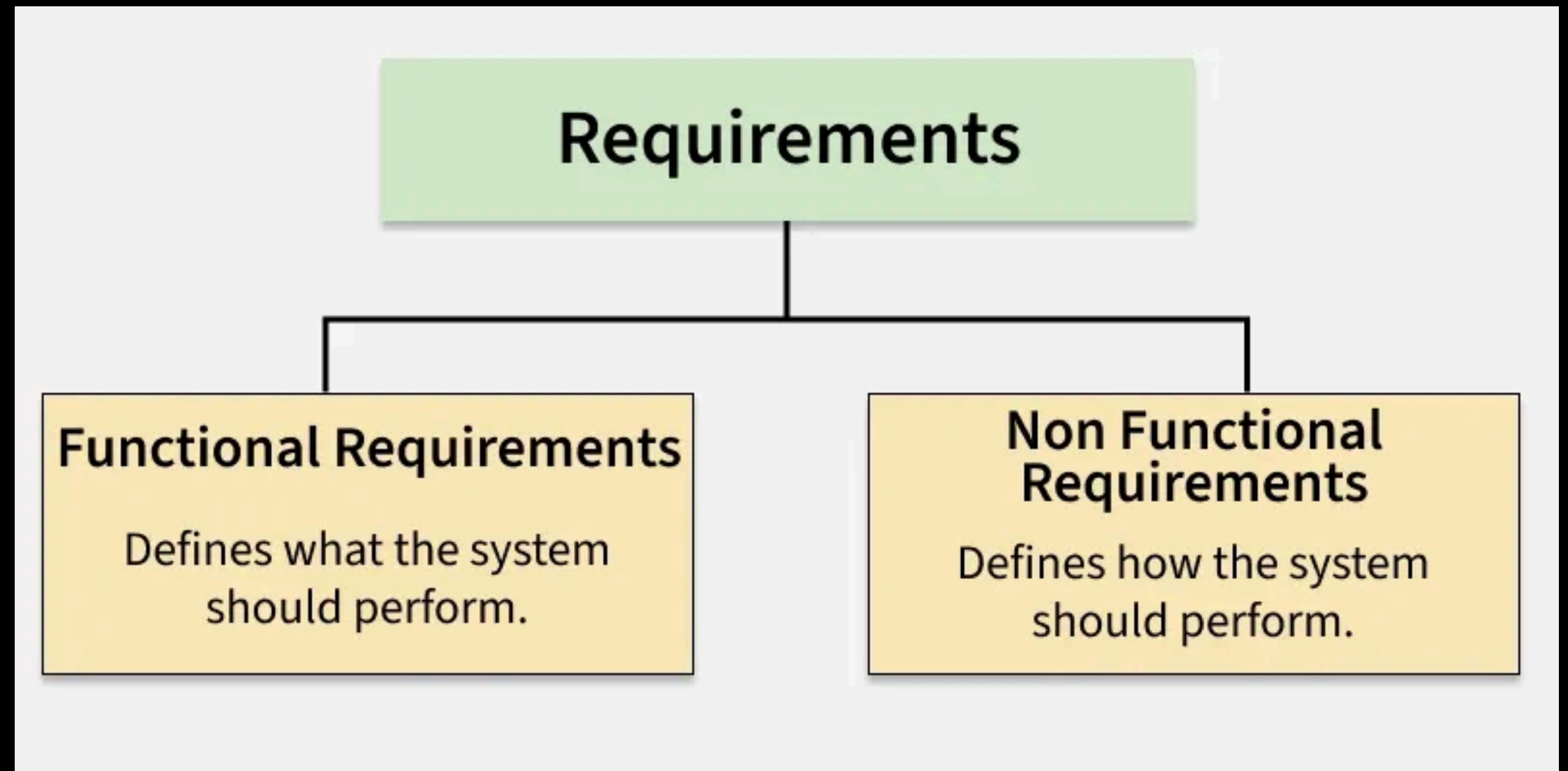
FR vs NFR

FR:

- What features should the system include?
- What edge cases must be considered in the design?

NFR:

- How fast should the system respond to user actions?
- How secure should it be against unauthorized access?



Good Requirements

atomic · achievable · cohesive · complete
current · independent · modifiable · traceable
unambiguous · verifiable · necessary

SOFTWARE PRODUCT QUALITY								
FUNCTIONAL SUITABILITY	PERFORMANCE EFFICIENCY	COMPATIBILITY	INTERACTION CAPABILITY	RELIABILITY	SECURITY	MAINTAINABILITY	FLEXIBILITY	SAFETY
FUNCTIONAL COMPLETENESS	TIME BEHAVIOUR	CO-EXISTENCE	APPROPRIATENESS	FAULTLESSNESS	CONFIDENTIALITY	MODULARITY	ADAPTABILITY	OPERATIONAL CONSTRAINT
FUNCTIONAL CORRECTNESS	RESOURCE UTILIZATION	INTEROPERABILITY	RECOGNIZABILITY	AVAILABILITY	INTEGRITY	REUSABILITY	SCALABILITY	RISK IDENTIFICATION
FUNCTIONAL APPROPRIATENESS	CAPACITY		LEARNABILITY	FAULT TOLERANCE	NON-REPUDIATION	ANALYSABILITY	INSTALLABILITY	FAIL SAFE
			OPERABILITY	RECOVERABILITY	ACCOUNTABILITY	MODIFIABILITY	REPLACEABILITY	HAZARD WARNING
			USER ERROR PROTECTION		AUTHENTICITY	TESTABILITY		SAFE INTEGRATION
			USER ENGAGEMENT		RESISTANCE			
			INCLUSIVITY					
			USER ASSISTANCE					
			SELF-DESCRIPTIVENESS					
iso25000.com								

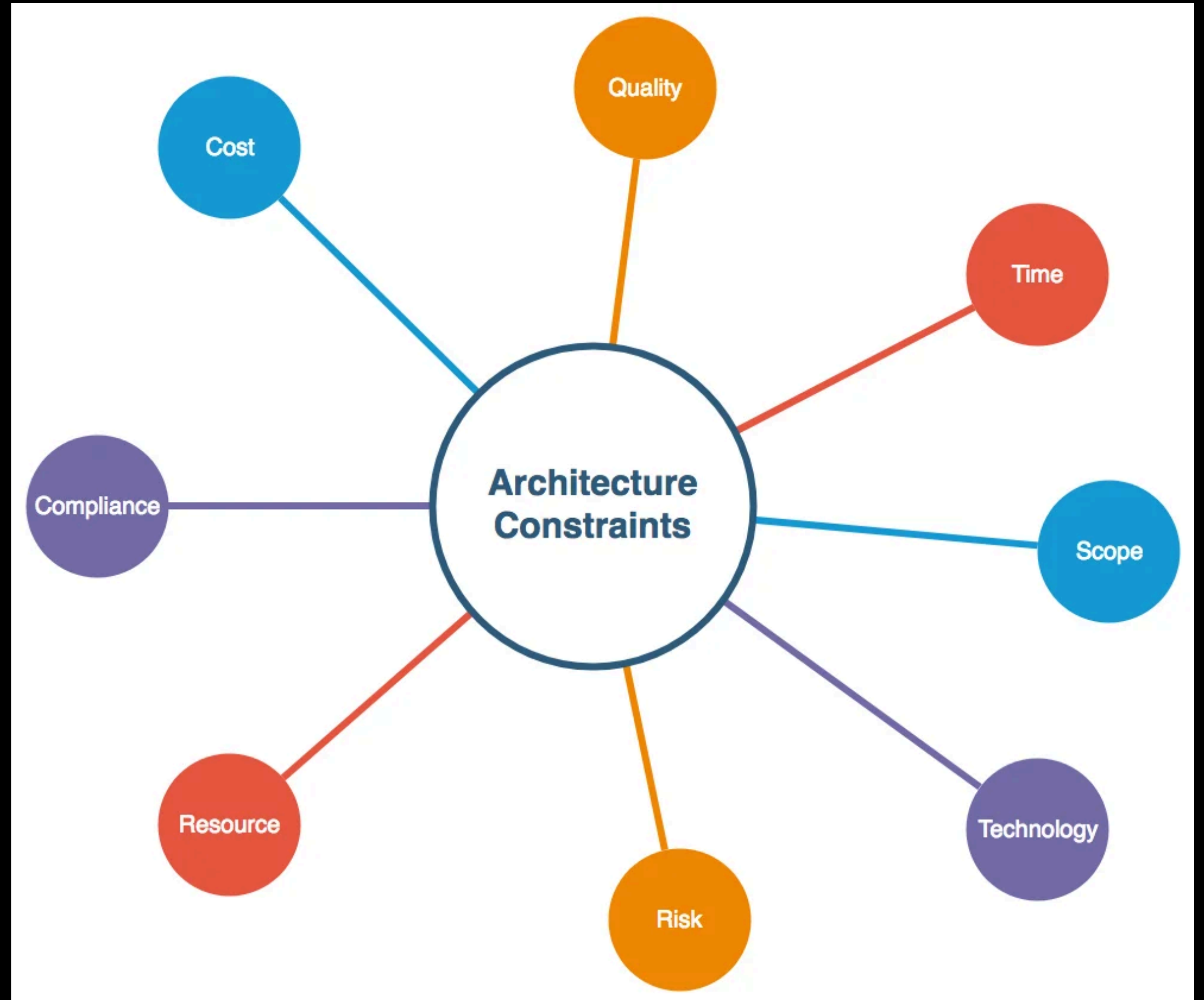
MoSCoW

the method of prioritisation



Architecture Constraints

are trade-offs which challenge solution design because they limit flexibility and innovation



practice